

w8L43 Function Approximation (38min)

Lookup Table - representation for value function

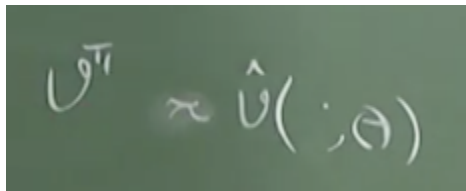
Bandit, $v(s)$, $q(s,a)$ - for every s we have values

Drawback:

- Memory
 - suppose memory is not an issue - just need large table
- Handling continuous state
 - Let us say we have discrete
- We are talking about state (v) and value (q) functions
 - Use some parameters
 - Unless I have visited that state few times 10^{25} states (during phd)
 - If I have to experience state multiple times - i wouldn't have graduated
- **We need to Generalise!**
- **Use Neural Networks**
 - AlphaGo Won against - highest ever human master!
 - Parameters are weights of the NN
- Lookup table is a para representation
 - It represents value $\rightarrow$ same no. of entries as look up table

Function Approximation = Parameterised Representation

- Accept Approximation
- On policy evaluation $\rightarrow$ It talks about control
- 10th chapter - is empty



Left of $\cdot$; argument of fn

What should be criteria for $\hat{v}$

1st stage: Decide what family of representation

- How output will depend on θ (some functional form - NN, para - weights)
- No. of layers in NN (if at google, msft - talk 150 layers)
- Regression Trees,
- Complex state aggregation - popular
- Linear parameterisation

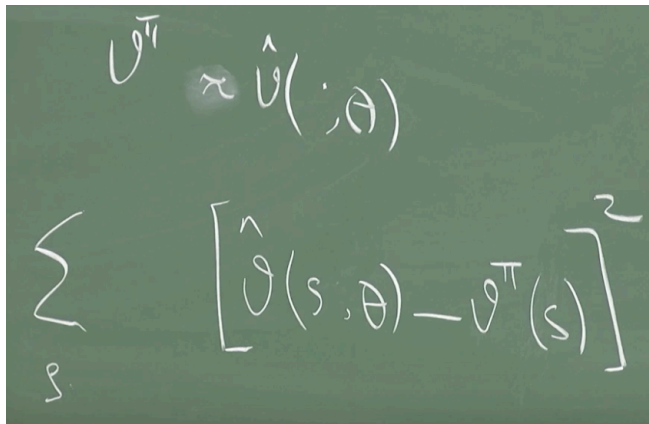
State is represented by $\mathbf{x}$ or y coordinate θ_1 , θ_2

2nd stage: what is the error measure - Performance measure

3rd: how will i find θ - that will give parameter

Performance is mean Square error

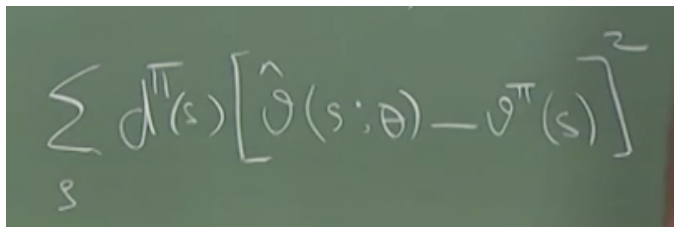
In Regression - Least squares is good enough


$$v^{\pi} \approx \hat{v}(s; \theta)$$
$$\sum_s [\hat{v}(s; \theta) - v^{\pi}(s)]^2$$

what is problem with 1/N

- Weighing each weight uniformly - good approximation everywhere
- Expected Error $\rightarrow$ weighted by prob of that occurring
- We can do Imprical mean $\rightarrow$ by using 1/N

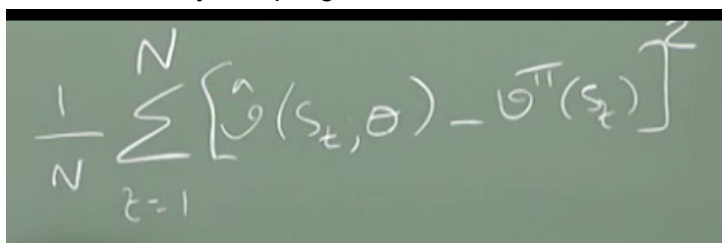
What will be the probability here


$$\sum_s d^{\pi}(s) [\hat{v}(s; \theta) - v^{\pi}(s)]^2$$

D-pi-s $\rightarrow$ steady state prob of state s under policy pi - distribution - markov chain - keep running

- Stop chain - when you come to the state - take action
- Dynamics looks like Markov chain

Used ON-Policy sampling


$$\frac{1}{N} \sum_{t=1}^N [\hat{v}(s_t; \theta) - v^{\pi}(s_t)]^2$$

Can i optimise and optimise theta $\rightarrow$ write differently

Target - estimate -> flipped it

$$= \frac{1}{N} \sum_{t=1}^N \left[v^{\pi}(s_t) - \hat{v}(s_t, \theta) \right]^2$$

Very serious problem:

- Don't know $v^{\pi}(s)$ -> i don't have target -> what we learn
- Learn to specify target
- We don't know v^{π}
- In classical we have x and $f(x)$

$$\text{Regr: } \langle x, f(x) \rangle \rightarrow \hat{f}(x)$$

RL:

- Here training data is trajectory $s_t, a_t, s_{t+1}, a_{t+1}, s_{t+2}, \dots$

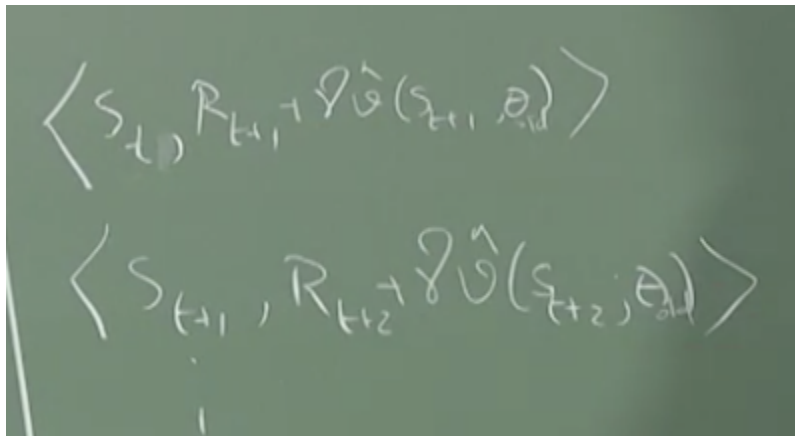
$$\begin{array}{l} \text{RL} \\ s_t, a_t, R_{t+1}, s_{t+1}, a_{t+1}, R_{t+2}, \dots \\ \downarrow \\ \hat{v}(s; \theta) \end{array}$$

Need to learn v^{π}

Create set of training data:

$$\begin{array}{l} \langle s_t, R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \rangle \\ \langle s_{t+1}, R_{t+2} + \gamma R_{t+3} + \dots \rangle \end{array}$$

Alternatively -> Create training data - for regression


$$\langle s_{t+1}, R_{t+1} + \gamma \hat{v}(s_{t+1}, \theta_{old}) \rangle$$
$$\langle s_{t+2}, R_{t+2} + \gamma \hat{v}(s_{t+2}, \theta_{old}) \rangle$$

theta-old

Seems little acceptable

- Set up non-stationary target
- theta -new will lead to new target

How can I use RL to training NN - as it is moving target

Jerry Toussaro -> fired up NN to learn BackGammon

With linear para we can show convergence

- **One important condition - Sampling ON policy**
-

It is counter intuitive - as soon i look at the data, data changes

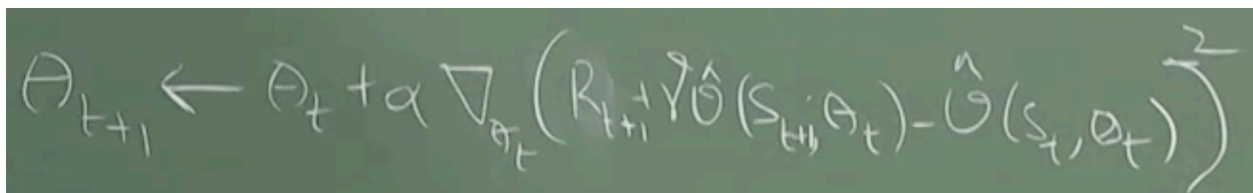
Gradient Descent/ Ascent to optimize function:

- Function we are dealing is error - go to opposite direction - MSE
-

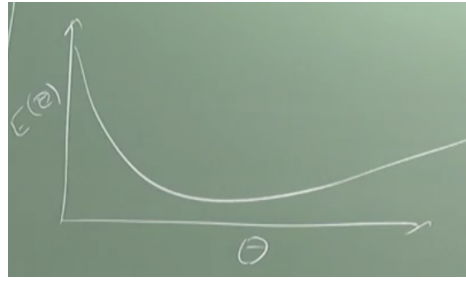
Popular - SGD -> single point estimate

- Have truly online-TD fn approximation

Calling at very state so call it theta-t


$$\theta_{t+1} \leftarrow \theta_t + \alpha \nabla_{\theta_t} (R_{t+1} + \gamma \hat{v}(s_{t+1}, \theta_t) - \hat{v}(s_t, \theta_t))^2$$

w8L44 Linear Parameterization (15min)

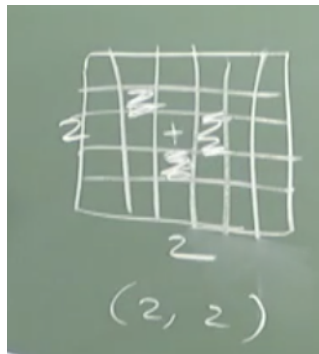


Error surface - what is x axis, y axis

$$S \rightarrow \phi_1(s), \phi_2(s), \dots, \phi_k(s)$$

S represented by k features

- What are the features - depend on problem - fi-1 x coordinate, fi-2 y axis
- Backgammon - no. of coins in position 1, 2
- Jerry Toussaro had - > 192 features



- GridWorld -> x, y coordinate, how many robots left of me, obstacles right of me
- Is obstacle below me, L, R or above me



going clockwise

Can take any problem

- What is the right encoding you should be using
- It depends!

Mathematically desirable property:

- Linearly Independent Vectors
- What are these vectors $\rightarrow$ all observations of ϕ_1, ϕ_2
- Write them one below each other
 - Columns k to be linearly independent
- For Regression Trees
 - What will happen if features are linearly dependent
 - Should not be catastrophic - others will be redundant
 - Doesn't matter
 - Stability - correctness will be fine
- $\phi(s)$ - whole vector evaluated at s = 1st column

- Capital $\phi(s)$ $\rightarrow$ dimension m

Linear parameterisation

Put $\hat{v}$ in θ_t

Take gradient of current estimate (NOT target)

For linear case:

For linear

$$\Theta_{t+1} \leftarrow \Theta_t + \alpha' (R_{t+1} + \gamma \hat{v}(s_{t+1}, \Theta_t) - \hat{v}(s_t, \Theta_t))$$

Anything else:

Gradient of v -hat will be -ve if : so became +

For linear

$$\Theta_{t+1} \leftarrow \Theta_t + \alpha' (R_{t+1} + \gamma \hat{v}(s_{t+1}, \Theta_t) - \hat{v}(s_t, \Theta_t)) \Phi(s)$$
$$\Theta_t + \alpha' \delta_t \Phi(s)$$

Essentially TD error

Talk about specific forms of Linear approximator

- Different choice of Φ

Linear para - different choices of Φ

For linear

$$\Theta_{t+1} \leftarrow \Theta_t + \alpha' (R_{t+1} + \gamma \hat{V}(s_{t+1}, \Theta_t) - \hat{V}(s_t, \Theta_t)) \Phi(s_t)$$

$$\Theta_t + \alpha' \delta_t \Phi(s_t)$$

Second thing:

Use of eligibility traces - backward view of TD lambda

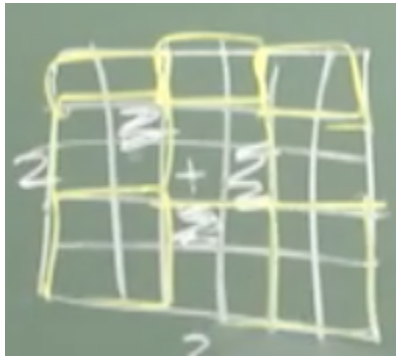
w8L45 State Aggregation Methods (22min)

Lookup tables - is a linear function approximation

V_i - is linear function

Another form of indicator function - state aggregation

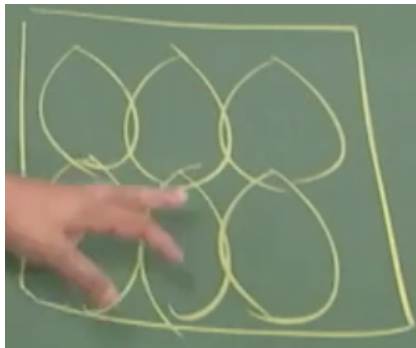
- Have very large state space million x million
- Trying to reduce dimensionality
- can assume - states that are close by have (10x10) grid will have same value
 - One of this 100 states
- Sample generating is ON-Policy



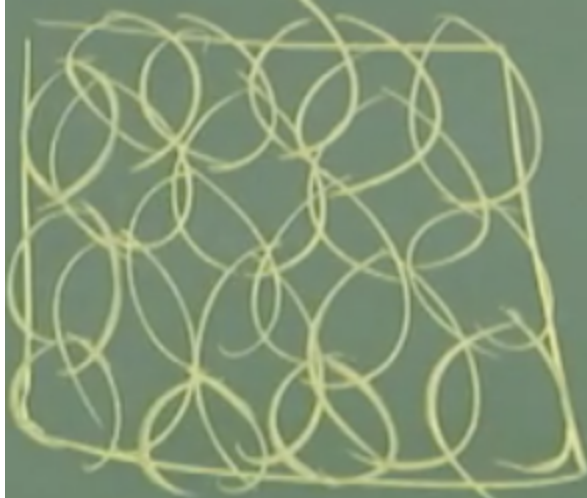
broken is smaller - now have only 9 indicator (instead of 30)

For Continuous state space - robot manipulation

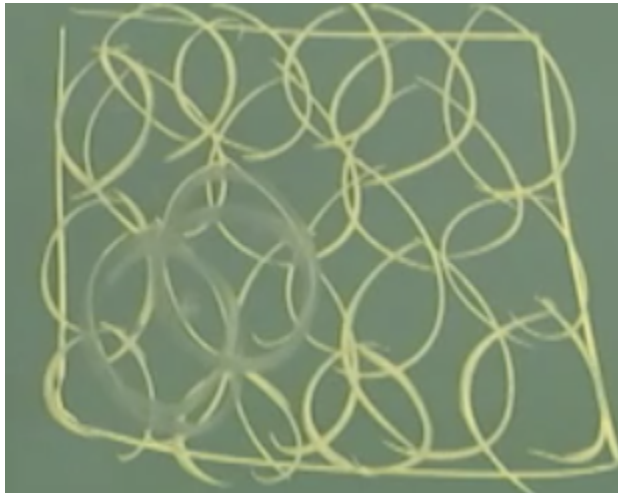
- Can talk about compact state



not covering whole space



But we have overlap !
Generalisation will be better! Will get updated together

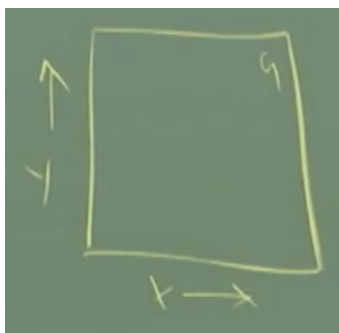


Coarse Coding

Not abrupt shift as in discrete space
It is desirable to have overlap (1, 2, 3, or 4 overlap)

All of these are geometric problem - nearness in nearness in value function

State $\rightarrow$ manifold $\rightarrow$ basis functions $\rightarrow$ nearness in transformed space is nearness in value fn



goal, para, x and y

How will value fn look like

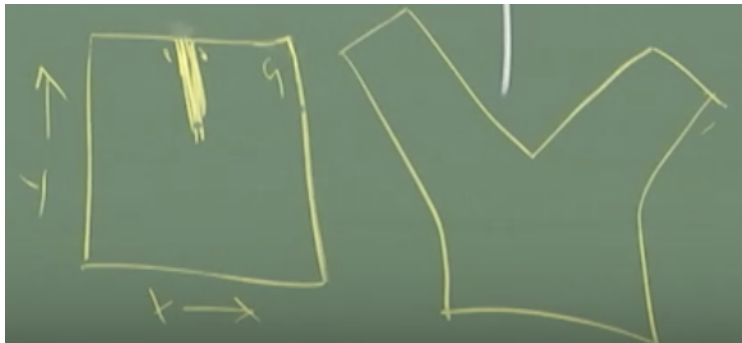
-ve direction peak will be at (0,0) will got to G - smooth plane stipped over

Now put a wall:

- There will be some discontinuity



one way to fix - embedding split the room open

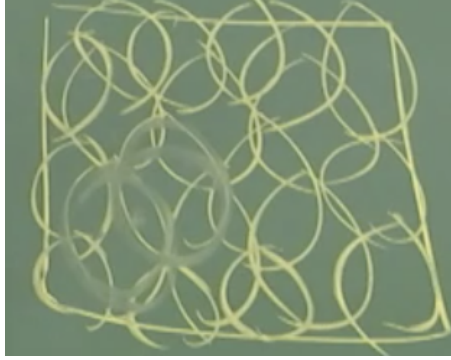


Quasi corridor

Now x, y coordinate - may be good approximation

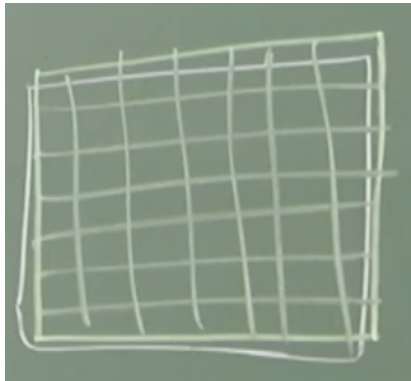


Y-coordinate is curve - turned left



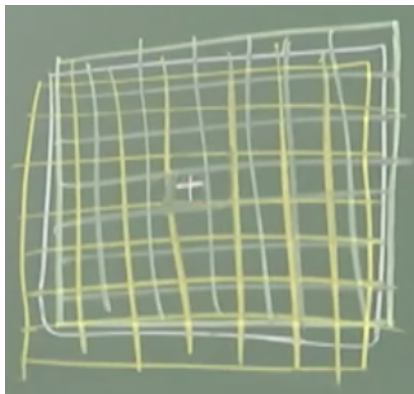
Coarse Coding

Tile Coding



doesn't give overlap

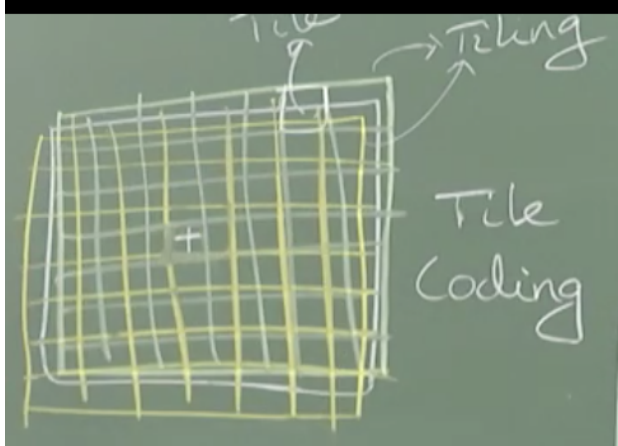
DO multiple such grids



Always light up 2 indicator functions - green to yellow,
Every state will have two indicator function

Use 4-7 tilings

How many indicator fn = no. of tiling



Good for many RL tasks

You can't plan Go, Backgammon - using tile coding

Deep Learning -> they find f_i from data -> Biggest Advantage

- Every thing is done in one shot -> f_i and value function

Downside:

- Significantly powerful computing
- Large data

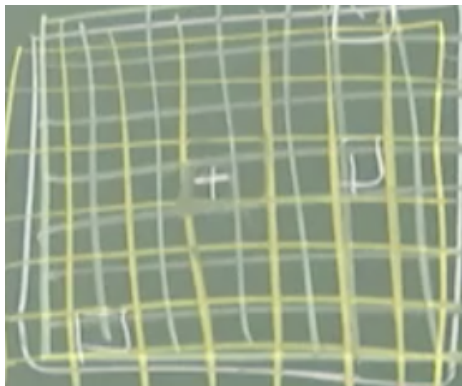
CMAC (Cerebral Model Actuation Controller) tile:

- Then do tricks to compress tiling - don't have to store vector θ
- Today you have memory so don't do compression

Don't have to store tile - we can have a functional form

- Need to store θ
- At the end of the day - it is a lookup table
-

Gives - un-intended Generalisation



one tile from left below, and one tile from right center

Could give same hash

- Destructive generalisation
- I don't like hashing
- In large state space - hashing is faster

w8L46 Function Approximation and Eligibility Traces (27min)

Eligibility traces - record of all states we have seen so far

What do we do in fn approx

- Previously eligibility trace was on s
- Should it be on θ or f_i ?
- θ - which entity needs its value to be updated ->
- How will f_i appear $f_i \delta$, will appear as $f_i e \delta$
- For linear case
 - f_i or θ both are fine
- In non-linear case
 - Have to take gradient wrt to θ
 - What does the gradient tell you - how much this θ is responsible for output
 - Right way to track eligibility
 - Keep track of gradient, δ is

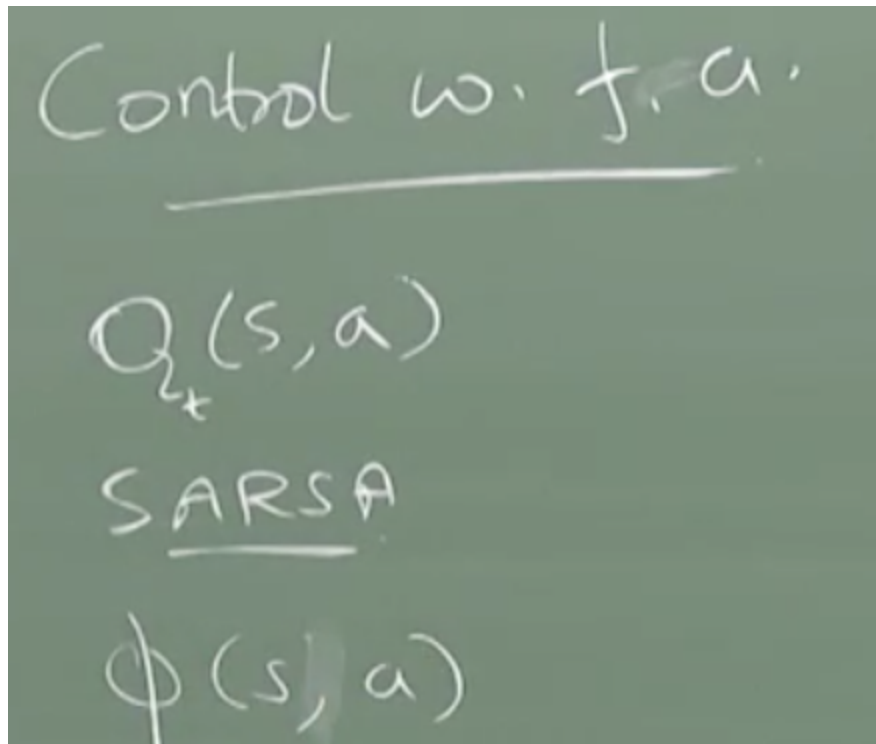
$$\begin{aligned}\delta_t &= R_{t+1} + \gamma V(s_{t+1}) - V(s_t) \\ \vec{e}_t &= \gamma \lambda \vec{e}_{t-1} + \nabla_{\theta_t} V(s_t) \\ \vec{\theta}_{t+1} &= \vec{\theta}_t + \alpha \delta_t \vec{e}_t\end{aligned}$$

For linear approx - δ will be f_i

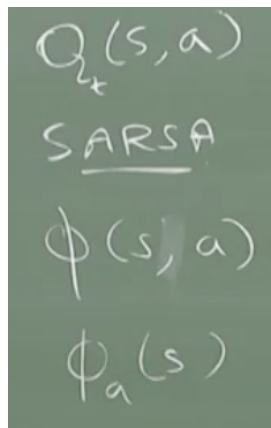
+1 for state visited - equivalent to TD(λ)

Control with function approximation:

- What we need to care about - keep track of - use sarsa or q-learning



Need encoding for a - we have small set of actions
 - MDPs have small connectivity - go to few states
 In that case - don't explicitly

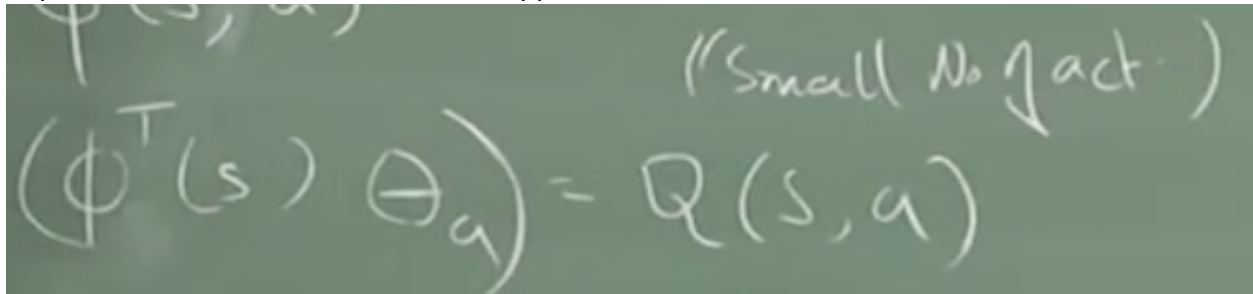


or

$$(\phi^T(s) \theta_a) = Q(s, a)$$

If i have 4 actions - will have 4 different approximation
 Will be trained in interlinked method
 $q(s', a')$ for (s, a) will use theta

Representation wise will be stored a 4 approximation


$$(\phi^T(s) \theta_a) = Q(s, a) \quad \text{("Small No of act.")}$$

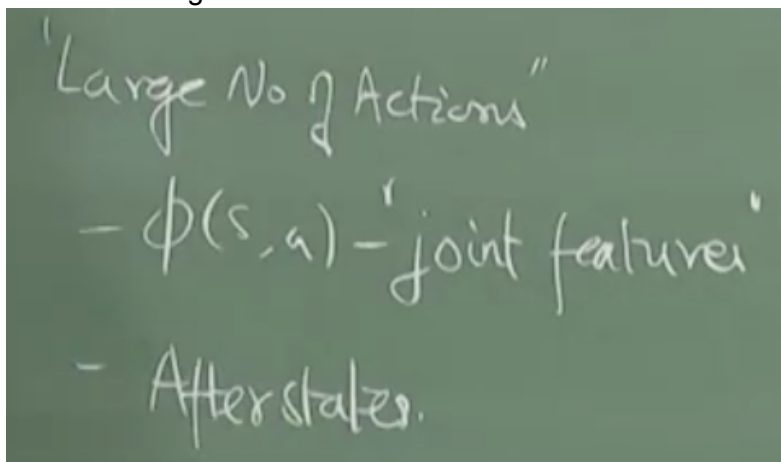
For large number of actions or continuous actions

Can generalise across actions - they may have similarity
You actually do $Q(s, a)$

Values of after state are in really some form of q function

- After state is using s and a
- Generalisation across similar (similar is more than same)
- Similar after state will have same value
- Allows you to generalise across actions
- Query on (s, a)

Other advantage:



'Large No of Actions'

- $\phi(s, a)$ - 'joint features'
- After states.

- Fewer parameters to estimate
- More theta to estimate - more data and trials are needed
- Memory is non-issue

We don't know what large is:

- 4 action - large could be 4 - after state
- Continuous - can't enumerate
 - Can i use afterstate
 - Possibly -> is variant of Depends
 - It has to be deterministic component to dynamics -> det forward model
 - State space can be define
-

Deterministic component/ moves - selling cow

- Natured components - growing cows - stochastics

You make move deterministic

Opponents move is stochastics

- If opponent's model is deterministic - add his state and plan ahead

Joint features:

In case of Bandit:

- Joint encoding of state and actions

The image shows handwritten text on a chalkboard. At the top, it says 'SARSA(λ)'. Below that, the equation $\vec{e}_t = \gamma \lambda \vec{e}_t + \nabla_{\theta_t} Q(s_t, a_t)$ is written.

For accumulating gradient

The image shows handwritten equations on a chalkboard. The first equation is $\delta_t = R_{t+1} + \gamma V(s_{t+1}) - V(s_t)$. The second equation is $\vec{e}_t = \gamma \lambda \vec{e}_{t-1} + \nabla_{\theta_t} V(s_t)$. The third equation is $\vec{\theta}_{t+1} = \vec{\theta}_t + \alpha \delta_t \vec{e}_t$. The fourth equation is $|\vec{e}_t| = |\vec{\theta}| \neq |s|$.

Don't want theta to be no. of space

1st two lengths - no. of component is e and theta. No. of state space

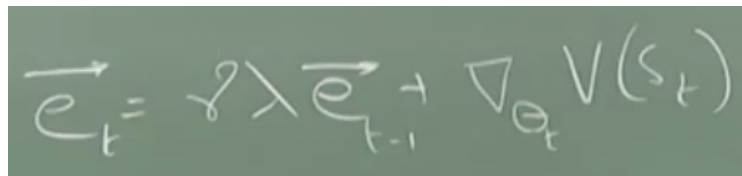
For Replacing version:

- You can't define in all case
- Only when it is indicator function (it could be k of n encoder)
- F_i should be 1 or zeros



$$\vec{e}_t = \gamma \vec{e}_{t-1} + \nabla_{\theta} V(s_t)$$

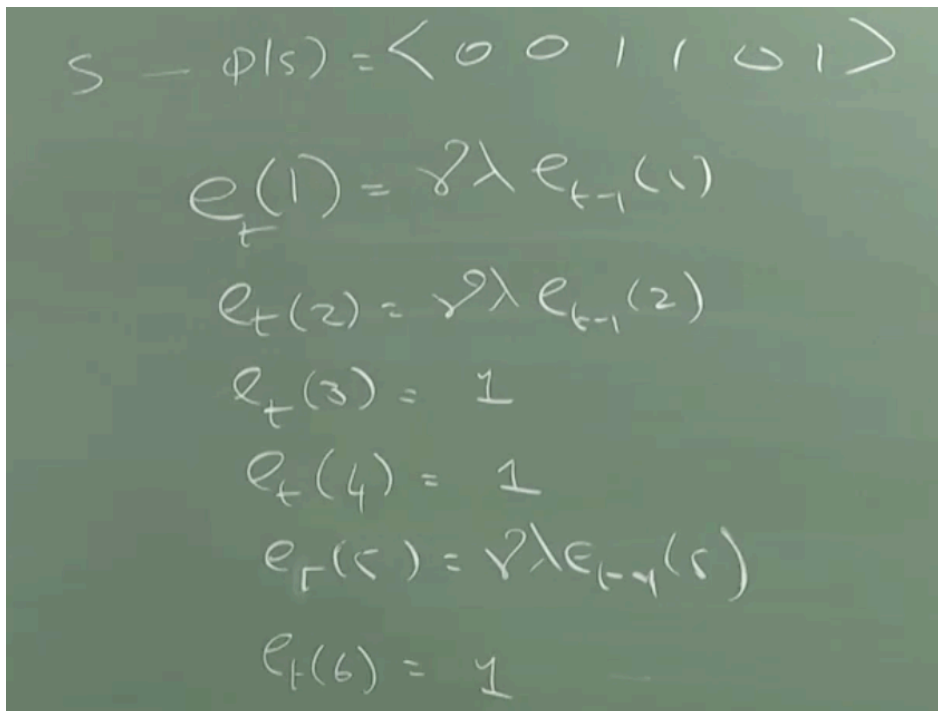
F_i is one - replace with 1



$$\vec{e}_t = \gamma \lambda \vec{e}_{t-1} + \nabla_{\theta} V(s_t)$$

When f_i is zero decay it:

For every state,



$$s - \phi(s) = \langle 0 \ 0 \ 1 \ 1 \ 0 \ 1 \rangle$$

$$e_t(1) = \gamma \lambda e_{t-1}(1)$$

$$e_t(2) = \gamma \lambda e_{t-1}(2)$$

$$e_t(3) = 1$$

$$e_t(4) = 1$$

$$e_t(5) = \gamma \lambda e_{t-1}(5)$$

$$e_t(6) = 1$$

For replacing traces

$$s - \phi(s) = \langle 0 \ 0 \ 1 \ 1 \ 0 \ 1 \rangle$$

$$e_t(1) = \gamma \lambda e_{t-1}(1)$$

$$e_t(2) = \gamma \lambda e_{t-1}(2)$$

$$e_t(3) = 1 + \gamma \lambda e_{t-1}(3)$$

$$e_t(4) = 1 + \gamma \lambda e_{t-1}(4)$$

$$e_t(5) = \gamma \lambda e_{t-1}(5)$$

$$e_t(6) = 1 + \gamma \lambda e_{t-1}(6)$$

For accumulating traces added values in box

0 do discounting

Non-zero - initialize to that value

We have seen - binary coding - can use replacing traces

w8L47 LSTD and LSTDQ (49min)

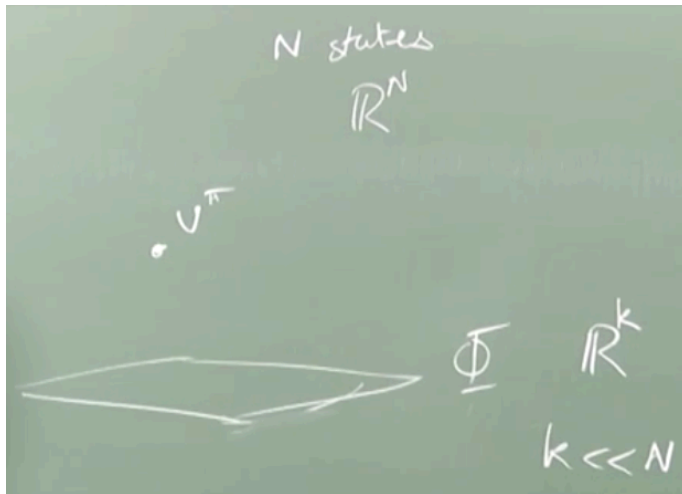
Used quite widely:

What exactly does linear fn approx does -

N states - v^π resides in $\mathbb{R}^N$ state space

1st coordinate - 1st state space

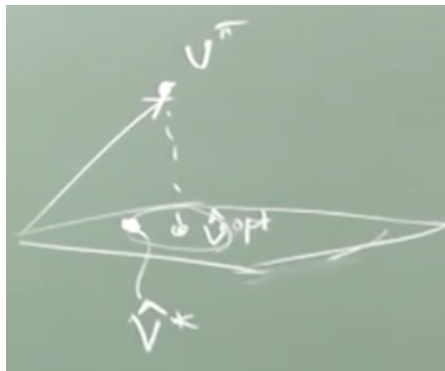
plane spanned by feature ϕ_i , - very small $\mathbb{R}^k$, want $k \ll N$



For drawing $N=3$, $k=2$:-)

Want $\hat{v}^\pi \rightarrow$ is closest projection to plane - we are looking for

Least sq sol fit



We have $\hat{v}^{\text{opt}}$ - will be least sq project $\rightarrow$ will not get but will converge to some $\hat{v}^*$ which is in certain bounded distance

Why you think this is happening?

- Min least square
- Ideally minimizer $\hat{v}^{\text{opt}}$
- **Non-stationary is problem - target keeps changing - using $\hat{v}$ as target**

If we knew true v -pi value

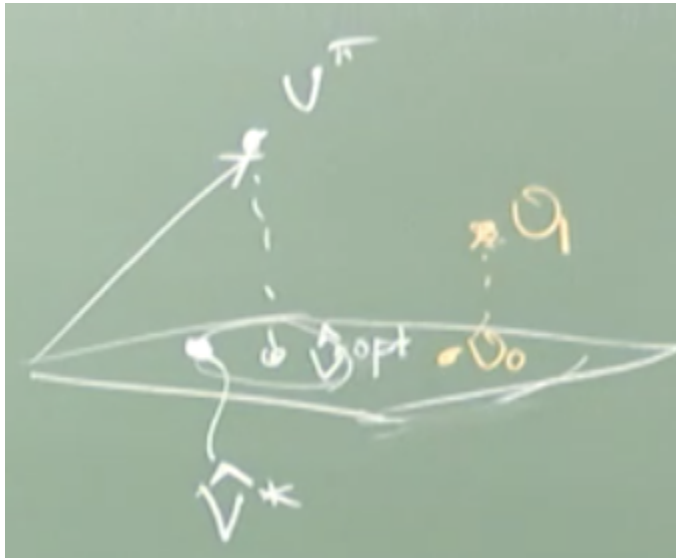
Given some mild condition $\mathbf{f}_i$ - can bound distance v -hat-star, v -hat-opt

Condition - linearly independently - ON-policy

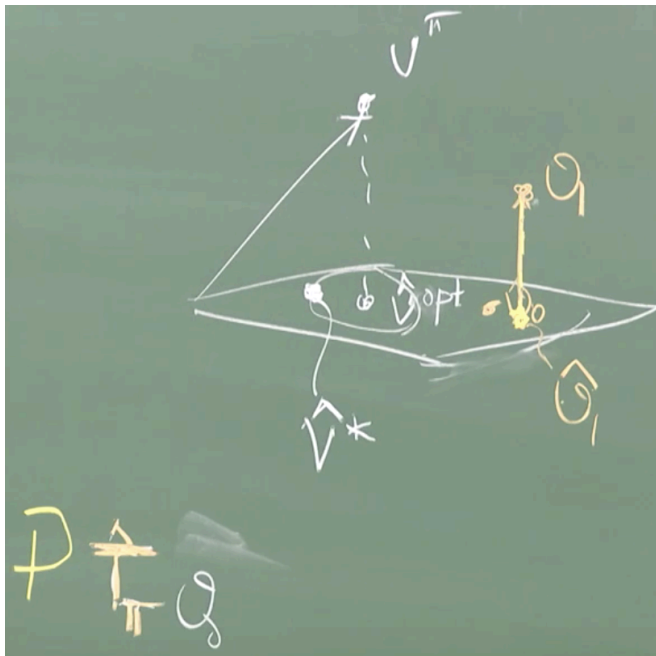
DP - every state is equally update - NOT ON-policy, will Diverge

Start off with v_0 -hat - somewhere in the space

- TD learning operator - sampled version T -hat-pi
- V_0 will not be here - apply T -pi on it
- Go to v_1



project it back



Yellow is projection matrix (P or row)

$$(P + T) \hat{q} \rightarrow \hat{v}_1$$

joint operation takes to v_1 -hat

You can think of composition operation - T then P?

Can we set it up as a proper regression format

- What is the matrix called - design matrix (all x s)

$$\langle (x_1, \dots, x_k), y \rangle_{s=1 \dots N}$$

If i want to something for value fn approximation

Should have some vector x

$$\langle (x_1, \dots, x_k), y \rangle_{s=1 \dots N}$$

$$\langle (\phi_1(s), \dots, \phi_k(s)), v(s) \rangle$$

Data should feed to regressor

N - samples

Don't know $v(s)$

$$\langle (\phi_1(s), \dots, \phi_k(s)), v(s) \rangle$$

$$v(s) = \underline{\Phi}^T(s) \theta$$

- Assume $v(s)$ comes from

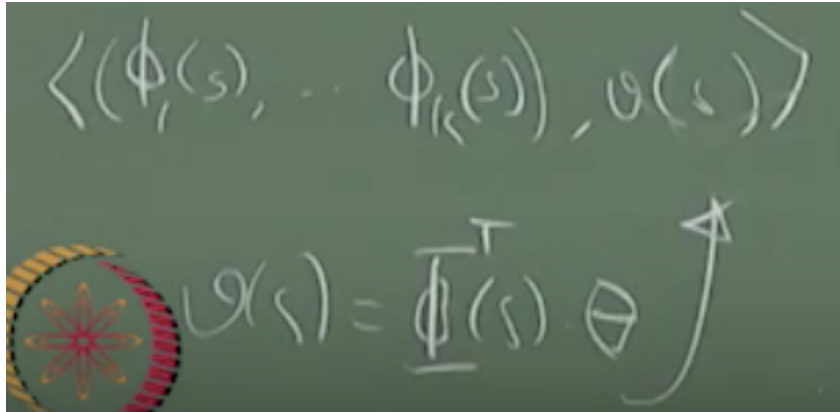
15min again:

Why uninteresting:

What will converge to MC fixed sample - least sq min of return - regression

TD - certainty equivalent

Make initial guess of theta ->

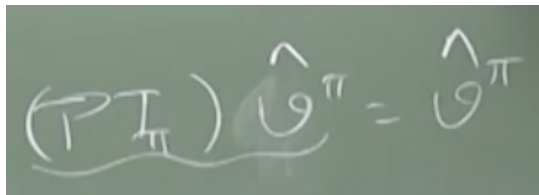

$$\langle (\phi_1(s), \dots, \phi_k(s)), \theta(s) \rangle$$
$$\theta(s) = \Phi^T(s) \theta$$

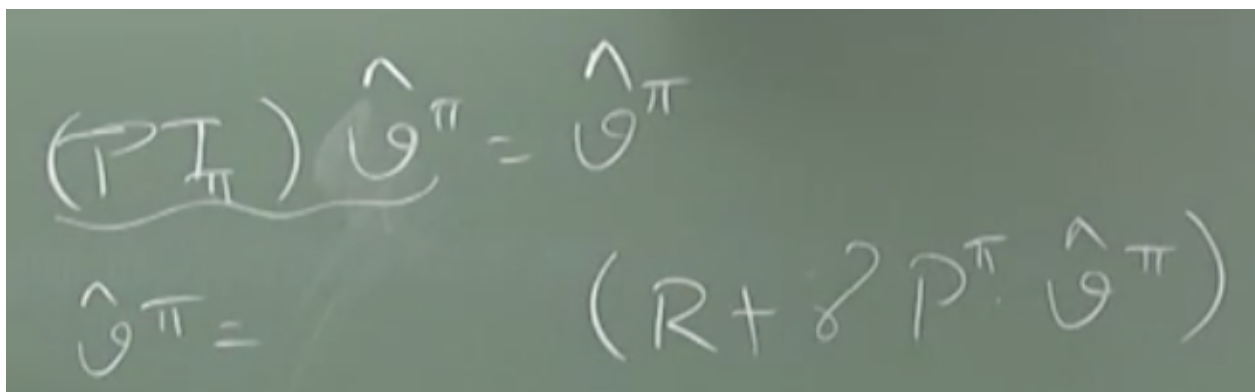
Keep doing this again and again

Input is in form of ϕ

Output is $\phi^T \theta$

Try to solve in some kind of - want to get to fixed point solution


$$(PI_\pi) \hat{\theta}^\pi = \hat{\theta}^\pi$$


$$(PI_\pi) \hat{\theta}^\pi = \hat{\theta}^\pi$$
$$\hat{\theta}^\pi = (R + \gamma P^\pi \hat{\theta}^\pi)$$

Look at the projection - how will it look like

$f_i(s)$ is one row
 F_i - is for all states

$$\begin{aligned} (P I_\pi) \hat{v}^\pi &= \hat{v}^\pi \\ \hat{v}^\pi &= \Phi (\Phi^T \Phi)^{-1} \Phi^T (R + \gamma P^\pi \hat{v}^\pi) \end{aligned}$$

Entire bellman equation - P^π
 Can write it for

$$\hat{Q}^\pi = \Phi (\Phi^T \Phi)^{-1} \Phi^T (R + \gamma P^\pi \hat{Q}^\pi)$$

F_i will be $f_i(s, a)$

$$\begin{aligned} (P I_\pi) \hat{v}^\pi &= \hat{v}^\pi \\ \hat{v}^\pi &= \Phi (\Phi^T \Phi)^{-1} \Phi^T (R + \gamma P^\pi \hat{v}^\pi) \\ \hat{Q}^\pi &= \Phi (\Phi^T \Phi)^{-1} \Phi^T (R + \gamma P^\pi \hat{Q}^\pi) \\ \Phi \Theta^\pi &= \Phi (\Phi^T \Phi)^{-1} \Phi^T (R + \gamma P^\pi \Phi \Theta^\pi) \end{aligned}$$

F_i -theta is entire matrix - on the fly - translation
 Replace $\hat{v}$ with $\hat{v}$ -theta- π

Each row in f_i is state

$$\begin{aligned}
 (P I_\pi) \hat{Q}^\pi &= \hat{Q}^\pi \\
 \hat{Q}^\pi &= \Phi (\Phi^T \Phi)^{-1} \Phi^T (R^\pi + \gamma P^\pi \hat{Q}^\pi) \\
 \hat{Q}^\pi &= \Phi (\Phi^T \Phi)^{-1} \Phi^T (R^\pi - \gamma P^\pi \hat{Q}^\pi) \\
 \Phi \Theta^\pi &= \Phi (\Phi^T \Phi)^{-1} \Phi^T (R^\pi - \gamma P^\pi \Phi \Theta^\pi) \\
 \Theta^\pi &= (\Phi^T (\Phi - \gamma P^\pi \Phi))^{-1} \cdot \Phi^T R^\pi
 \end{aligned}$$

Note - replaced R with R-pi everywhere

This give least square solution

- It treats every sample towards contributing to error
- Not right thing to do
-

Express states which are more important

Need Weighted Least Square -> introduce weight

Weighted Version.

$$\Phi^T W (\Phi - \gamma P^\pi \Phi) \Theta^\pi = \Phi^T W R^\pi$$

W over all state

- Usually w is - Steady state prob distribution

Weighted Version.

$$\underbrace{\underline{\Phi}^T \omega (\underline{\Phi} - \gamma \mathcal{P}^T \underline{\Phi})}_{A} \underline{\Theta}^T = \underbrace{\underline{\Phi}^T \omega R^T}_{b}$$

$$A \cdot \underline{\Theta}^T = b$$

What A and b are doing?

We are not doing incremental updates

For fixed set of samples - get a fit - don't know how good the fit is

W - should be steady state distribution

- We don't know it
- Approximate - from counting samples you have
- Can be very far away from w you want (for fixed set of samples)

On-policy - will be closer to v-hat-star

Why would you want to chose something else?

- You don't know the steady state distribution
- You use distribution from samples you have drawn

What is A:

$$A = \underline{\Phi}^T \omega (\underline{\Phi} - \gamma \mathcal{P}^T \underline{\Phi})$$

$$= \sum_s \phi(s) \cdot \omega(s) \cdot \left(\bar{\Phi}(s) - \gamma \sum_{s'} P(s, \pi(s), s') \cdot \phi(s') \right)^T$$

Moved from matrix notation to individual componentwise notations

$$\begin{aligned}
 A &= \underline{\Phi}^T \underline{W} (\underline{\Phi} - \gamma \underline{P}^T \underline{\Phi}) \\
 &= \sum_s \phi(s) \cdot \omega(s) \cdot \left(\bar{\Phi}(s) - \gamma \sum_{s'} P(s, \pi(s), s') \cdot \phi(s') \right)^T \\
 &= \sum_s \omega(s) \cdot \sum_{s'} P(s, \pi(s), s') \left[\phi(s) (\phi(s) - \gamma \phi(s'))^T \right]
 \end{aligned}$$

Added P so that can take it inside the inner summation

We can write as some kind of weighted some of this quantity

$$\begin{aligned}
 A &= \underline{\Phi}^T \underline{W} (\underline{\Phi} - \gamma \underline{P}^T \underline{\Phi}) \\
 &= \sum_s \phi(s) \cdot \omega(s) \cdot \left(\bar{\Phi}(s) - \gamma \sum_{s'} P(s, \pi(s), s') \cdot \phi(s') \right)^T \\
 &= \sum_s \omega(s) \cdot \sum_{s'} P(s, \pi(s), s') \underbrace{\left[\phi(s) (\phi(s) - \gamma \phi(s'))^T \right]}
 \end{aligned}$$

I can keep sampling - from s to s' - compute - take average - weighted, will give A

$$\underline{\theta}^\pi = \left(\underline{\Phi}^T (\underline{\Phi} - \gamma \underline{P}^T \underline{\Phi}) \right)^{-1} \cdot \underline{\Phi}^T \underline{R}^\pi$$

How can we solve regression - when we have f_i and R here

If i know the model, i will solve it directly

Figure out f_i and R

We have way to find out A matrix

Given samples of the form - L such samples

$$\mathcal{D} = \{ (s_i, a_i, r_i, s'_i) \mid i = 1, 2, \dots, L \}$$

It is i-th sample, nothing to do with time index

Form estimate of A^π

$$\tilde{A} = \frac{1}{L} \sum_{i=1}^L \left[\phi(s_i) (\phi(s_i) - \gamma \phi(s'_i))^T \right]$$

$$\tilde{A} = \frac{1}{L} \sum_{i=1}^L \left[\phi(s_i, a_i) (\phi(s_i, a_i) - \gamma \phi(s'_i, \pi(s'_i)))^T \right]$$

Action version of A^π

What about b ?

$$b = \Phi^T W R^\pi$$
$$= \sum_s \phi(s) \cdot w(s) \cdot \sum_{s'} P(s, \pi(s), s') \cdot R(s, \pi(s), s')$$

$$b = \Phi^T W R^\pi$$
$$= \sum_s \phi(s) \cdot w(s) \cdot \sum_{s'} P(s, \pi(s), s') \cdot R(s, \pi(s), s')$$
$$= \sum_s w(s) \sum_{s'} P(s, \pi(s), s') \cdot \left[\phi(s) \cdot R(s, \pi(s), s') \right]$$

$$\hat{b} = \frac{1}{L} \sum_{i=1}^L \phi(s_i) \cdot r_i$$

$$\hat{b} = \frac{1}{L} \sum_{i=1}^L \phi(s_i, a_i) \cdot r_i$$

Action version

LSTD - Least square TD (for value function)

- Use estimated $\tilde{A}$, $\tilde{b}$

-

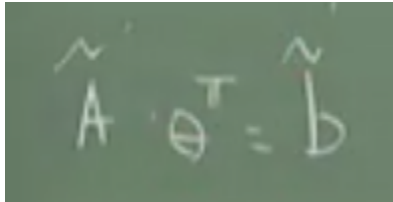
-

LSTD-q:

When you operate with q fn it is called LStd-q

LSTD(lambda):

Will have gamma and lambda term in A matrix



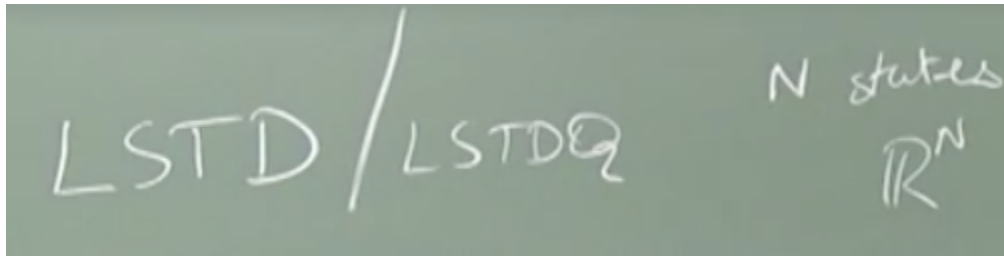
A handwritten equation on a chalkboard: $\hat{A} \cdot \theta^T = \hat{b}$. The symbols are written in white chalk on a dark green background.

$\tilde{A}$ is invertible $\rightarrow$ linearly independent

1/L will go away, what are you left - just summation, keep adding to A matrix

- Incrementally add

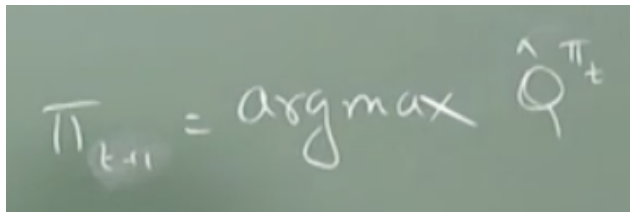
w8L48 LSPI-FittedQ (17min)



Handwritten text on a chalkboard: LSTD / LSTDQ N states $\mathbb{R}^N$

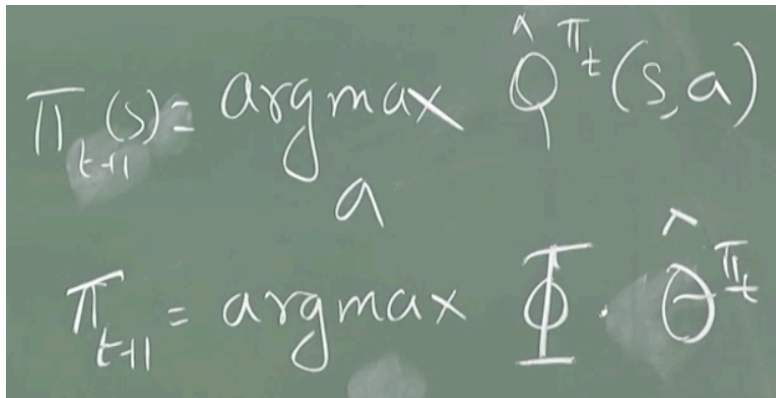
Found something for policy, what is next -> Control

Take policy, find a q fn, what is the tricky part?



Handwritten equation on a chalkboard: $\pi_{t+1} = \operatorname{argmax} \hat{Q}^{\pi_t}$

arg max is done statewise



Handwritten equations on a chalkboard:

$$\pi_{t+1}(s) = \operatorname{argmax}_a \hat{Q}^{\pi_t}(s, a)$$
$$\pi_{t+1} = \operatorname{argmax} \Phi \cdot \hat{\Theta}^{\pi_t}$$

Simplify

What are the problems?

- How state and actions are represented
- What does it mean max

Set it up as linear program - many ways you can solve

From 2003 - solve it for few states

$$\pi_{t+1} = \underset{\pi}{\operatorname{argmax}} \Phi \cdot \hat{\Theta}^{\pi_t}$$

$$\begin{aligned} s_1, \dots, s_d &= s \\ a_1, \dots, a_d &= \pi_t(s) \end{aligned}$$

s_1 to s_j

Have actions a_1 to a_j

$$\begin{aligned} a_1 &= \phi_1(s_1) \dots \phi_k(s_1) \left(\begin{array}{c} s_1, \dots, s_d \\ a_1, \dots, a_d \end{array} \right) = \pi_t(s) \\ \vdots \\ a_j &= \phi_1(s_j) \dots \phi_k(s_j) \end{aligned}$$

Give this as training data to a classifier - have it solve the problem

A_1 to a_j - finite action case - classifier

A_1 to a_j - infinite - regression

We are using q fn to recover the policy

Represent π - as function approximation

$$\pi_{t+1} = \underset{a}{\operatorname{argmax}} \Phi \cdot \hat{\Theta}^T \pi_t$$

This is the problem i really want to solve

- S is continuous state space
- Then it becomes hard - to get closed form solution
- Precompute and keep it as policy
- We do max as we have action
- I want to precompute
- Find best actions for a_1 to a_j

$$\phi_1(s_1) \dots \phi_{12}(s_1)$$

is input - encoding of state

$$a_1 \dots \phi_1(s_1) \dots \phi_{12}(s_1)$$

a_j is output

Will learn some state - f is rep of policy π_i

f_i - need not be same, i can use different f_i

- Have to use different set of features

Least square Policy Iteration (LSPI)

$$\pi_{t+1}(s) = \underset{a}{\operatorname{argmax}} \hat{Q}^{\pi_t}(s, a) \quad \text{LSPI}$$

$$\pi_{t+1} = \underset{a}{\operatorname{argmax}} \Phi \cdot \hat{\Theta}^{\pi_t}$$

$$\begin{matrix} a_1, & \phi(s_1), & \dots, & \phi_k(s_1), & \dots, & \phi_d(s_1) \\ \vdots & & & & & \\ & & & & & \end{matrix} \begin{pmatrix} s_1, & \dots, & s_d \\ a_1, & \dots, & a_d \end{pmatrix} = \pi_{t+1}(s)$$

$$\phi_1(s_d), \dots, \phi_k(s_d)$$

Refer - paper

Fitted q iteration:

You do q learning

- Start with fixed set of samples
- Generate targets $\rightarrow v - \pi_i$, i q learning $R + \max$ over
- With same data - train fn approx to approx target
- Didn't put any linear constraint
- Training data dependent on q fn output

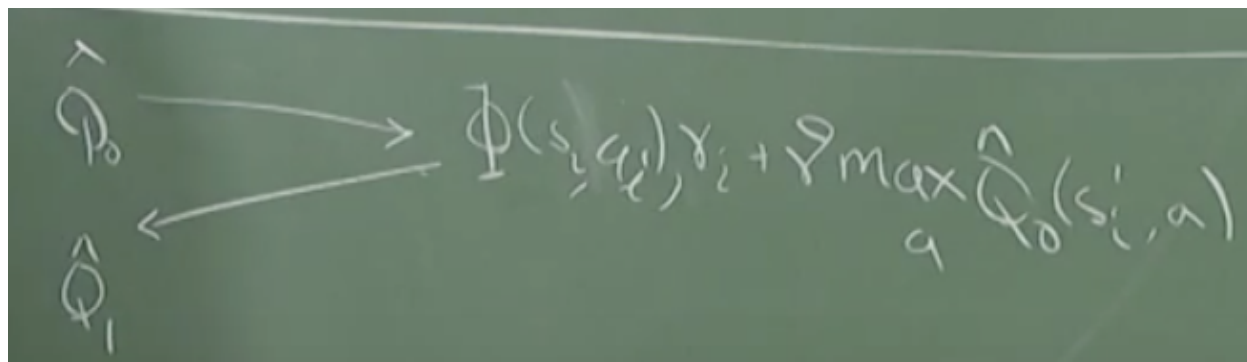
Q-hat to be given with, training data

$$\Phi(s_i, a_i) r_i + \gamma \max_a \hat{Q}_0(s'_i, a)$$

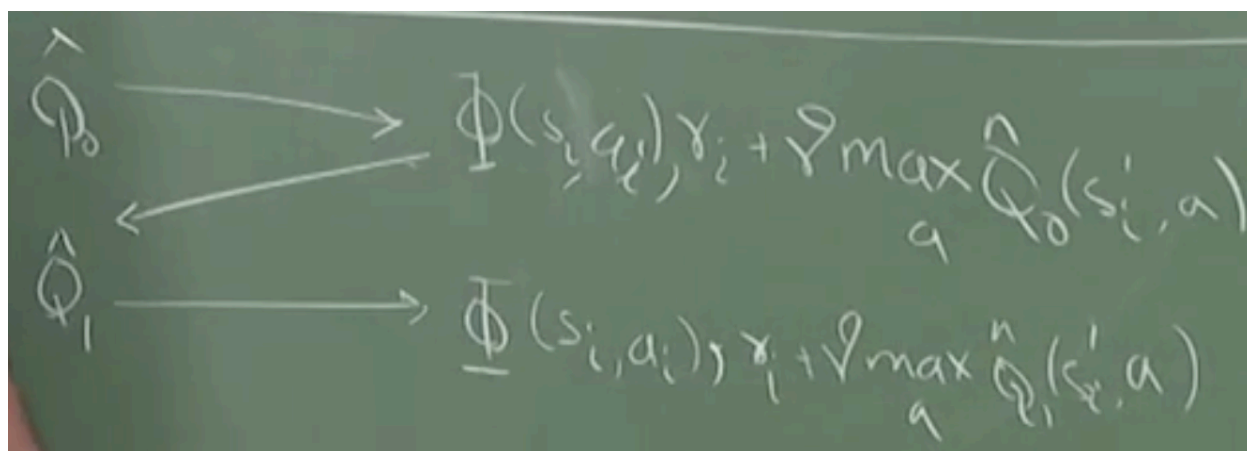
Do for all i-s

$$\mathcal{D} = \{ (s_i, a_i, r_i, s'_i) \mid i = 1, 2, \dots, L \}$$

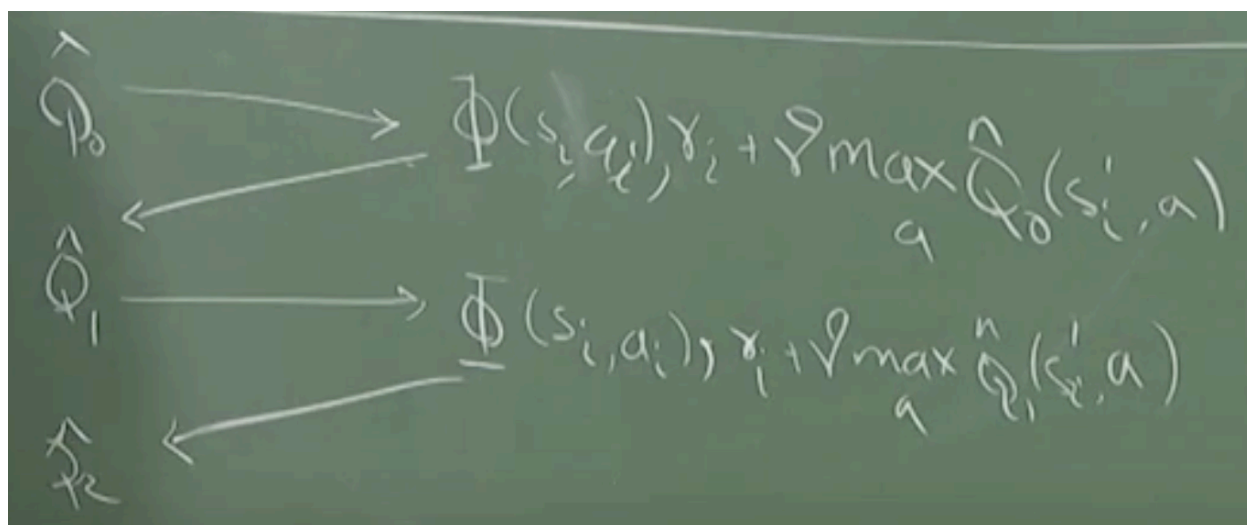
Data is of this form
Feed to fn approx



Will give q1-hat



Create that



Solve that - to give q2-hat

Keep going - iterative - q-learning - fitting a function to it - fitted q iteration

Have to be careful

- Should have sufficiently large samples to find max

$$\mathcal{D} = \{ (s_i, a_i, r_i, s'_i) \mid i = 1, 2, \dots, L \}$$

From this trajectory - generating data

If you have fixed set of sample - no way of generating

- Fitted q iteration is better

In Deep Learning - fitted q learning is used